

HuBMAP - Hacking the Human Vasculature

535521 Selected Topics in Visual Recognition using Deep Learning

徐葆騏、吳念澤、江怜儀、王好瑄

Group14

GitHub: https://github.com/benhsu0828/2026version_HuBMAP_Hacking_the_Human_Vasculature.git

1 Introduction

This project studies the Kaggle competition *HuBMAP – Hacking the Human Vasculature* [1]. The task is instance segmentation: for each 512×512 histology tile, the model must output separate object masks with confidence scores. The final evaluation follows a mean Average Precision (mAP) style metric over instance masks, so both localization quality and mask quality matter.

The training data is heterogeneous. Dataset 1 contains the cleanest annotations and is used as the main validation and fine-tuning source in our experiments. Dataset 2 contains more weak/noisy annotations and is useful for increasing data coverage, but it can also inject label noise. Dataset 3 provides additional whole-slide image (WSI) regions; in our implementation it is treated mainly as extra WSI context in the dataset metadata, while the main training pipelines focus on Dataset 1 and Dataset 2. This setting makes cross-validation design important: neighboring tiles from the same WSI can be visually similar, so random tile-level splits would overestimate generalization.

2 Top Solutions Analysis

We do not attempt to survey all leaderboard solutions. Instead, we focus on two ideas that directly influenced our implementation.

2.1 First-place direction: WSI-aware heterogeneous ensembles

The first-place Kaggle write-up frames the problem as box-quality-driven instance segmentation: AP is dominated by bounding-box localization, while mask prediction can be delegated to a strong mask head after the boxes are fixed [2]. This explains why the public release is not a single semantic-segmentation model. It is a heterogeneous MMDetection ensemble that includes RTMDet-X, YOLOX-X, and Mask R-CNN streams. The released training repository also marks RTMDet-X as the best single model, and its configs add mask supervision through an extra mask head on top of the detector [2].

Several design choices are especially relevant:

- **WSI-aware validation.** Dataset 1 is split by spatial location instead of random tile splitting. The write-up reports that this split is harder than random splitting, which makes it more useful for avoiding over-optimistic validation.
- **EMA as a stabilizer.** The author highlights Exponential Moving Average (EMA) as the most important technique in the experiments. Multiple EMA copies were evaluated from a single training run, which gave a stable basis for model selection.
- **Mask supervision for better boxes.** RTMDet-X already produced strong boxes, but adding mask supervision and random rotation improved bbox mAP. The mask head was therefore used not only for final masks, but also as an auxiliary signal for localization.
- **Box-first ensemble.** The final ensemble uses WBF over three RTMDet models, one YOLOX-X model with mask supervision, and one Mask R-CNN. All five model types are represented by two different fold weights. The final masks are generated from a Mask R-CNN mask head at input size 1440, and the write-up explicitly notes that no TTA was used.

This project therefore uses the first-place pipeline as the main baseline: reproduce the MMDetection ensemble behavior, keep the bbox-first interpretation of the metric, then test whether a new stream improves or hurts the fused box set.

2.2 Third-place direction: multi-stage training

The third-place write-up focuses on how to properly use noisy annotations [3]. The final system uses five MMDetection-based models: two ViT-Adapter-L models, one CBNetV2-Base model, one DetectoRS ResNeXt-101-32x4d model, and one DetectoRS ResNet-50 model. The author reports using only competition data, with no external image data. The key training idea is multi-stage learning: first pretrain on all noisy Dataset 2 WSIs for a short schedule with light augmentation and high learning rate, then fine-tune on clean Dataset 1 for more epochs with heavier augmentation, higher image resolution, and a lower learning rate.

Multi Stage Approach

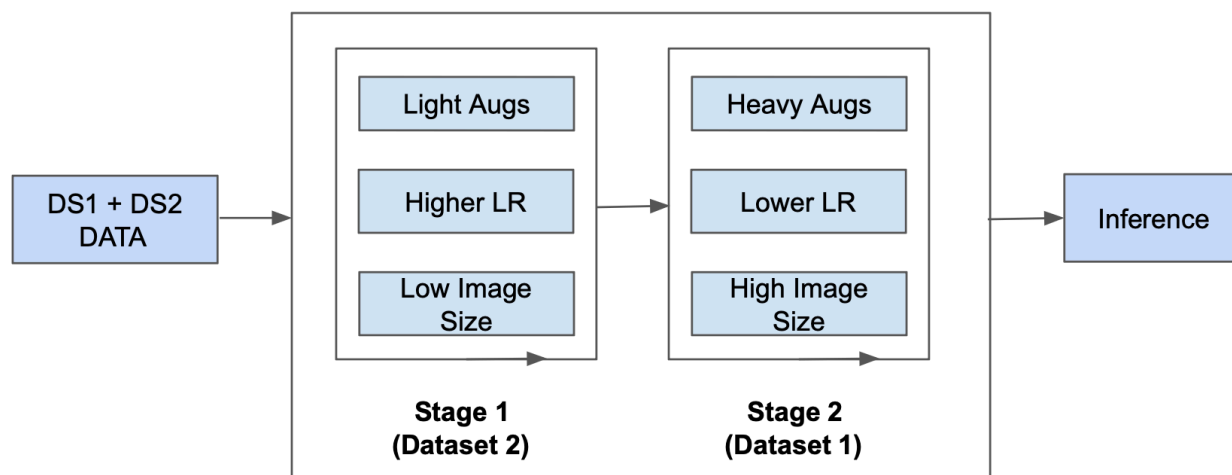


Figure 1: Third-place multi-stage training diagram from the Kaggle write-up [3]. The central idea is to use Dataset 2 for broad noisy pretraining and Dataset 1 for clean high-resolution fine-tuning.

The same write-up also describes a pseudo-label extension for Dataset 3. In that branch, multi-stage models generate predictions, WBF ensembles them, confidence thresholding filters them, and the remaining pseudo labels are combined with Dataset 3 before another multi-stage training pass. The author states that pseudo labels were used in some final ensemble models, although they did not clearly improve leaderboard score. Reported useful techniques include multi-stage training, flip TTA, higher mask-head loss weights for HTC-based models, SGD, WBF, and erosion followed by one dilation step as post-processing.

Multi Stage Pseudo Label Approach

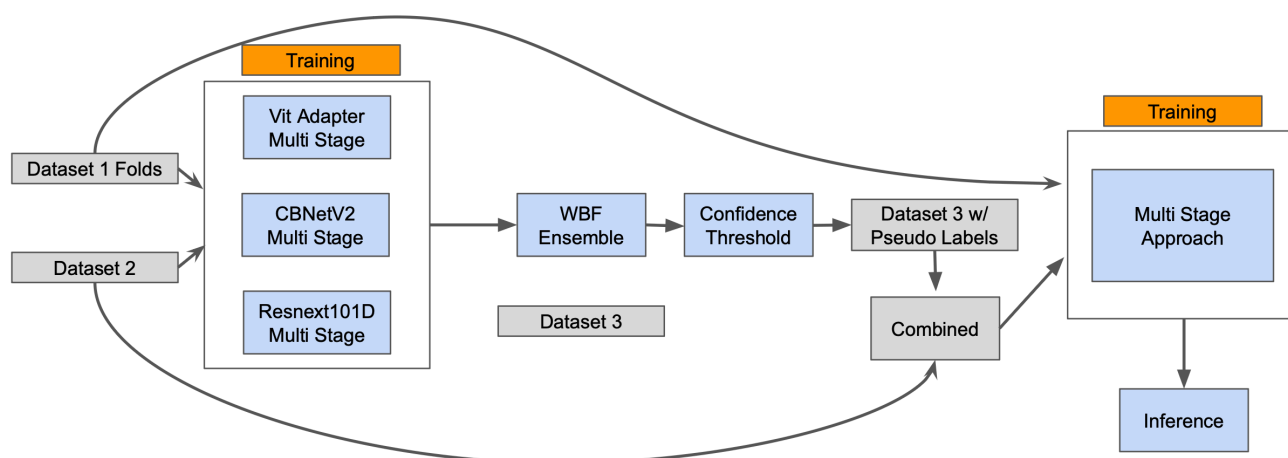


Figure 2: Third-place pseudo-label extension from the Kaggle write-up [3]. Ensemble predictions are thresholded into Dataset 3 pseudo labels and then folded back into another multi-stage training pass.

In our code this appears in `mmdet_v3_solution`: Stage 1 trains on Dataset 2, and Stage 2 fine-tunes on Dataset 1. This strategy is reasonable because Dataset 2 can teach broad morphological priors, while Dataset 1 should correct boundary precision and class noise. However, the original third-place system used much larger high-capacity backbones, higher training and inference resolutions such as 1400–2048 pixels, multi-scale inference, and a five-model ensemble. Our experiment replaces that family with an InternImage-T backbone inside Cascade Mask R-CNN. This makes the experiment informative but not a faithful reproduction of the third-place architecture.

3 Baseline Reproduction

3.1 Baseline A: first-place MMDetection ensemble

The reproduced baseline follows the public first-place release. Its inference notebook runs ten MMDetection streams:

$$\{r0i, r1i, s0i, s1i, m0i, m1i, y0i, y1i, sb0i, sb1i\}.$$

Each stream outputs bounding boxes, scores, and labels. These boxes are fused by Weighted Boxes Fusion (WBF) [4]. The fused boxes are then passed into a Mask R-CNN style mask branch from `m1i` to produce final instance masks. The local workflow notes record this legacy MMDetection baseline as reproducing a Kaggle score of about 0.589.

For comparison, the original first-place author reported 0.589 private and 0.317 public. Our closest first-place-only rerun used the released first-place models with TTA and scored 0.577 private and 0.316 public. Therefore, the reproduced baseline is close on public score but about 0.012 lower on private score.

3.2 Data processing

For MMDetection experiments, annotations are converted from `polygons.jsonl` to COCO format. Dataset 1 is split spatially within WSI 1 and WSI 2 to create `dtrain0i.json` and `dval0i.json`. Dataset 2 is exported as `dtrain_dataset2.json` for pre-training.

For YOLO experiments, we use a separate preprocessing pipeline. Each 512×512 tile is padded to 768×768 by stitching neighboring tiles when available. Neighbor polygons are shifted into the padded coordinate frame, which reduces truncation artifacts for vessels crossing tile boundaries. The output is written in Ultralytics YOLO segmentation format with four spatial folds. The split uses Dataset 1 for validation and excludes spatially overlapping Dataset 2 tiles from the corresponding training fold to reduce leakage.

4 Experiment 1: YOLO26-Seg Ensemble

4.1 Motivation

The first experiment asks whether adding a modern YOLO segmentation model can improve the first-place ensemble. The hypothesis is straightforward: YOLO26-Seg is architecturally different from Cascade/Mask R-CNN style models, so it may add useful diversity. If its errors are complementary, WBF should produce better boxes and therefore better final masks.

4.2 Method

We trained a four-fold YOLO26x-Seg ensemble using Ultralytics. The training script uses:

- input size 768×768 , matching the padded tile size;
- AdamW with fixed learning rate 10^{-3} ;
- heavy geometric augmentation: 180° rotation, flips, scale, mosaic, mixup, and copy-paste;
- class filtering at inference so that only `blood_vessel` instances are submitted.

The YOLO stream is inserted into the original notebook between the MMDetection inference cells and the WBF cell. Each YOLO fold produces `yolo0.pkl` to `yolo3.pkl`, using the same dictionary format as MMDetection:

$$\text{pred_instances} = \{\text{bboxes}, \text{scores}, \text{labels}\}.$$

This makes YOLO a drop-in WBF stream. In the modified WBF cell, the four YOLO streams are assigned weights $[2, 2, 2, 2]$ and can be enabled with `ABLATION_MODE = baseline`.

4.3 Results and analysis

The YOLO-only validation results were reasonable for a lightweight independent stream, but they did not beat the reproduced first-place ensemble on private leaderboard. The best local fold-level YOLO mask mAP@50:95 ranged from 0.137 to 0.215, as shown in Table 1. The full Kaggle submission records are summarized in Table 2.

Fold	Best epoch	Mask mAP@50	Mask mAP@50:95
0	76	0.226	0.137
1	59	0.255	0.160
2	76	0.305	0.215
3	59	0.302	0.203

Table 1: Best local validation metrics of the YOLO26x-Seg folds from `runs/seg/fold*/results.csv`.

Submission setting	Inference / fusion details	Private	Public
Original first-place author	Public Kaggle write-up score	0.589	0.317
First-place models only	Released first-place models with TTA inference	0.577	0.316
YOLO26-Seg only	Four-fold YOLO26, direct inference, ensemble mask-NMS	0.420	0.387
YOLO26-Seg only	Four-fold YOLO26, 4-way TTA {orig, hflip, vflip, rot90}, WBF and weighted mask average	0.515	0.446
YOLO26-Seg + MedSAM-2	YOLO prompts with MedSAM-2 LoRA and vendored MedSAM2	0.258	0.287
First-place + YOLO26-Seg	10 first-place streams plus 4 YOLO streams, WBF weights [2, 2, 2, 2, 1, 1, 1, 1, 2, 2, 2, 2, 2]	0.572	0.304
First-place + YOLO26-Seg	Same 14-stream WBF with YOLO weights reduced to [0.5, 0.5, 0.5, 0.5] for the last four streams	0.563	0.308
First-place + YOLO26-Seg	Explicit <code>streams=14</code> implementation with YOLO weights [2, 2, 2, 2]	0.572	0.304

Table 2: Kaggle submission scores from the tested inference variants.

These records clarify the gap between the experiments and the earlier report draft. YOLO26-Seg improved strongly when moving from direct mask-NMS to 4-way TTA with WBF and weighted mask averaging, rising from 0.420 to 0.515 private. However, adding YOLO streams into the first-place WBF did not improve the first-place system: the best 14-stream result was 0.572 private, below both our first-place-only TTA rerun at 0.577 and the author’s 0.589. Lowering YOLO weights to 0.5 improved public score slightly but reduced private score to 0.563. The MedSAM-2 refinement branch also underperformed at 0.258 private, suggesting that prompt-driven mask refinement was not robust to the YOLO box distribution in this setup.

The negative result is still useful. First, YOLO did not provide enough diversity: many mistakes overlapped with the existing detector ensemble, especially on ambiguous vessel-like structures. Second, its bounding boxes were not consistently better than the Cascade/Mask R-CNN family. Since the final pipeline re-predicts masks from fused boxes, box quality dominates the final mAP. Third, the YOLO-only WBF/TTA variant looked attractive on public score (0.446 versus 0.316 for our first-place-only TTA rerun), but it was still weaker on private score (0.515 versus 0.577). Therefore, the public leaderboard alone was not reliable for selecting the final system, and adding a segmentation stream is not sufficient; the stream must improve private-set localization diversity.

5 Experiment 2: Two-stage InternImage Cascade

5.1 Motivation

The second experiment follows the multi-stage idea from high-ranking solutions: first learn from the larger noisy Dataset 2, then fine-tune on the cleaner Dataset 1. Instead of reproducing the original large ViT/CBNet/DetectoRS-style system, we implemented a more practical MMDetection v3 model that can run in our environment and be packaged for Kaggle.

5.2 Method

The model is Cascade Mask R-CNN [5] with:

- **Backbone:** InternImage-T with pure PyTorch DCNv3 operators;
- **Neck:** FPN for multi-scale feature fusion;

- **Head:** 3-stage Cascade RoI head with progressively stricter IoU thresholds;
- **Mask branch:** FCN mask head with mask loss weight increased to 2.0;
- **Resolution:** 1024×1024 training images.

Stage 1 trains on `dtrain_dataset2.json` for 10 epochs with light augmentation and AdamW learning rate 2×10^{-4} . Stage 2 loads the best Stage 1 checkpoint and fine-tunes on `dtrain01.json` for 20 epochs with stronger augmentation, including random flips, AutoAugment color/geometric policies, and Albumentations elastic transform. The Kaggle inference package uses FP16, six-view TTA, morphological post-processing, and offline MMDetection dependencies.

5.3 Results and analysis

Stage 1 reached local `coco/segm_mAP` 0.130 at epoch 8 on `dval01`. After fine-tuning on Dataset 1, Stage 2 improved substantially and reached 0.440 local `coco/segm_mAP` at epoch 12. This confirms that the Dataset2 $\rightarrow$ Dataset1 schedule is useful. However, the result still did not surpass the reproduced first-place ensemble, and it is below what one would expect from a faithful third-place-style system.

Model / stage	Data	Best epoch	BBox mAP	Segm mAP
InternImage Cascade Stage 1	Dataset 2	8	0.181	0.130
InternImage Cascade Stage 2	Dataset 1	12	0.407	0.440

Table 3: Local MMDetection validation metrics on `dval01`. These are not directly comparable to Kaggle private leaderboard scores.

The main reason is architecture mismatch. The referenced multi-stage solution direction relies on very large transformer-style backbones and detector ensembles. Our InternImage-T model is much smaller and is a different system. A second reason is Dataset 2 noise: a compact model may absorb noisy masks more directly, whereas a larger pretrained backbone can be more robust. Finally, our implementation used only a single InternImage Cascade stream. The first-place baseline derives much of its strength from ensembling and from using a dedicated high-resolution mask re-prediction stage.

6 Discussion

The two experiments show that leaderboard-level instance segmentation is a systems problem, not only a model-choice problem. The first-place pipeline works because it separates responsibilities: many models propose boxes, WBF filters and averages localization evidence, and a strong mask branch converts fused boxes into final masks. This structure also explains why the YOLO experiment did not help the final ensemble. YOLO26-Seg produced a reasonable standalone result after WBF/TTA (0.515 private, 0.446 public), but adding it to the first-place streams reduced the private score from our first-place-only 0.577 to 0.572.

The InternImage experiment gives a different lesson. Multi-stage training is useful: Stage 2 improved the local segmentation mAP from 0.130 to 0.440. But borrowing only the training schedule is not enough to reproduce a top solution. Backbone capacity, resolution, label-noise robustness, and ensemble integration all interact. InternImage-T was attractive because it was easier to package and run, especially with a pure PyTorch DCNv3 implementation, but this practical choice also limited its ceiling.

Overall, our best direction remains the reproduced first-place MMDetection ensemble. The YOLO and InternImage experiments are valuable ablations: YOLO tested whether simple diversity improves WBF, while InternImage tested whether Dataset2 pretraining transfers to Dataset1 fine-tuning. The submission gap is now clear: the original author reported 0.589 private, our first-place-only TTA rerun reached 0.577, and the best YOLO-augmented first-place ensemble reached 0.572. Future improvements should therefore focus on better box proposals, higher-capacity backbones, stronger WSI-aware validation, and more careful fusion rather than simply adding more segmentation outputs.

7 Conclusion

We reproduced the structure of the HuBMAP first-place solution and explored two extensions. The original first-place score was 0.589 private / 0.317 public, while our first-place-only TTA rerun reached 0.577 / 0.316. YOLO26x-Seg alone improved from 0.420 / 0.387 with direct mask-NMS to 0.515 / 0.446 with 4-way TTA, WBF, and weighted mask averaging, but inserting YOLO into the first-place WBF reduced private score to 0.572. The YOLO + MedSAM-2 branch was worse at 0.258 / 0.287. A two-stage Cascade Mask R-CNN

with InternImage-T confirmed the benefit of Dataset2 pretraining followed by Dataset1 fine-tuning, but the smaller backbone and single-model setup remained below the reproduced ensemble baseline. The project therefore supports a central conclusion: for this competition, high-quality box localization and ensemble design are more important than mask sharpness alone.

References

- [1] HuBMAP and Kaggle. HuBMAP - hacking the human vasculature. <https://www.kaggle.com/c/hubmap-hacking-the-human-vasculature>, 2023.
- [2] tascj. 1st place solution. <https://www.kaggle.com/c/hubmap-hacking-the-human-vasculature/discussion/429060>, 2023. Kaggle competition write-up.
- [3] Nischay Dhankhar. 3rd place solution: How to properly utilise noisy annotations. <https://www.kaggle.com/c/hubmap-hacking-the-human-vasculature/discussion/430242>, 2023. Kaggle competition write-up.
- [4] Roman Solovyev, Weimin Wang, and Tatiana Gabruseva. Weighted boxes fusion: Ensembling boxes from different object detection models, 2019.
- [5] Zhaowei Cai and Nuno Vasconcelos. Cascade R-CNN: Delving into high quality object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6154–6162, 2018.